EXP-1

AIM:

To develop a predictive text system using NGram and HMM (Hidden Markov Models) to predict the next word based on previous context.

A) NGram Model:

Program:

```
import nltk
import re
from nltk.corpus import brown
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter


nltk.download('brown')
nltk.download('punkt')
nltk.download('punkt_tab')


sentences = brown.sents()
text = " ".join([" ".join(sent) for sent in sentences])


text = text.lower()
text = re.sub(r'[^a-z\s]', '', text)
tokens = word_tokenize(text)
tokens = [t for t in tokens if t.strip() != ""]

unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))

unigram_freq = Counter(unigrams)
bigram_freq = Counter(bigrams)

vocab_size = len(set(tokens))


def predict_next_word_ngram(word):
    word = word.lower()

    candidates = {
        w2: count
```

```
        for (w1, w2), count in bigram_freq.items()
        if w1 == word
    }

    if not candidates:
        return "No prediction available"

    probs = {
        w: (count + 1) / (unigram_freq[(word,)] + vocab_size)
        for w, count in candidates.items()
    }

    return max(probs, key=probs.get)
user_word = input("Enter a word (N-Gram): ")
print("Predicted next word:", predict_next_word_ngram(user_word))
```

Output:

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a word (N-Gram): they are
Predicted next word: No prediction available
```

B) HMM (Hidden Markov Models)

Program:

```
import nltk
import random
from nltk.corpus import brown
from nltk.tag import hmm


nltk.download('brown')
nltk.download('universal_tagset')


tagged_sentences = brown.tagged_sents(tagset='universal')


trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(list(tagged_sentences[:5000]))


def predict_next_word_hmm(sentence):
    words = sentence.lower().split()
    tagged = hmm_model.tag(words)

    last_tag = tagged[-1][1]

    next_tag = max(
        hmm_model._states,
        key=lambda tag: hmm_model._transitions[last_tag].prob(tag)
    )

    word_probs = {
        word: hmm_model._outputs[next_tag].prob(word)
        for word in hmm_model._outputs[next_tag].samples()
    }

    top_words = sorted(
        word_probs.items(),
        key=lambda x: x[1],
        reverse=True
    )[:5]

    return random.choice(top_words)[0]


user_sentence = input("Enter a sentence (HMM): ")
print("Predicted next word:", predict_next_word_hmm(user_sentence))
```

Output:

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
Enter a sentence (HMM): they
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335: RuntimeWarning: overflow encountered in cast
  O[i, k] = self._output_logprob(si, self._symbols[k])
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:333: RuntimeWarning: overflow encountered in cast
  X[i, j] = self._transitions[si].logprob(self._states[j])
Predicted next word: was
```

Result:

Thus we successfully developed a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context.